

Contents

1	Attention, Transformer	2
1.1	Attention Layers	2
1.1.1	Previous architectures	2
1.1.2	Attention layer	3
1.1.3	Self-attention-idea	3
1.1.4	Self-attention	3
1.1.5	Self-attention step by step	3
1.1.6	Self-attention in matrix form	4
1.1.7	Attention matrix	4
1.1.8	Other score functions	4
1.1.9	Complexity	4
1.1.10	Multi-Head Attention — idea	5
1.1.11	Multi-Head Attention	5
1.1.12	Multi-Head Attention — step by step	5
1.1.13	Multi-Head Attention	5
1.2	Transformer Model	6
1.2.1	Transformer model overview	6
1.2.2	Encoder-encoder attention	6
1.2.3	Decoder-decoder attention	6
1.2.4	Encoder-decoder attention	7
1.2.5	Other operations	7
1.2.6	Positional encoding	7
1.2.7	Transformer size	7
1.2.8	Training details	8
1.2.9	Training details	8
1.2.10	Output generation: greedy search	8
1.2.11	Output generation: beam search	8
1.2.12	BLEU metric for NMT	9
1.3	Metrics and Inference for sequence generation	9
1.4	Transformer-based LMs and contextualized word embeddings: ELMO, BERT, GPT	9
1.5	Transformer for Computer Vision	9

Language Models

Autoregressive Modelling

$$p(x) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \dots p(x_T|x_1 \dots x_{T-1})$$
$$\langle \text{BOS} \rangle \rightarrow \text{RNN} \rightarrow \text{Linear} \rightarrow \text{Logits} \in \mathbb{R}^{|v|} \rightarrow \text{softmax} \rightarrow p(x|x_0 = \langle \text{BOS} \rangle)$$

$$\text{sample } x_1 \rightarrow \text{RNN} \rightarrow \text{Linear} \rightarrow \text{Logits} \rightarrow p(x|x_0 = \langle \text{BOS} \rangle, x_1 = x_1)$$

$$\text{sample } x_2 \rightarrow \text{RNN} \rightarrow \text{Linear} \rightarrow \text{Logits} \rightarrow p(x|x_0 = \langle \text{BOS} \rangle, x_1 = x_1)$$

$\langle \text{EOS} \rangle$ — End of Sequence.

Training a Language Model

θ — model parameters

$$p(x_t|x_0, \dots, x_{t-1})$$

$X = (x_1, \dots, x_T)$ — real sequences

$$p_\theta(x) = p_\theta(x_0)p_\theta(x_1|x_0) \dots p_\theta(x_T|x_0, x_1, \dots, x_{T-1})$$
$$\parallel$$
$$1$$

$$-\sum_{t=1}^T \log p_\theta(x_t|x_0, \dots, x_{t-1}) \rightarrow \min$$

1 Attention, Transformer

1.1 Attention Layers

1.1.1 Previous architectures

1) Fully-connected

- Fixed window
- A lot of weights

2) Convolutional

- Local dependencies

3) Recurrent

- Slow training (autoregressive)
- Troubles with gradients

1.1.2 Attention layer

- Look at all inputs to learn more global dependencies
- Share weights to work with variable length window
- Do not use autoregressive structure to make training parallelizable

1.1.3 Self-attention-idea

We update an embedding of each token using all input tokens:

- Find tokens relevant to the current one (compute relevance scores)
- Compute the weighted combination of their embeddings

$$p = \text{softmax}(k_1^T q_i, k_2^T q_i, \dots, k_l^T q_i)$$

$$\tilde{x}_i = \sum_{j=1}^L p_j V_j$$

1.1.4 Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$X \in \mathbb{R}^{L \times d} \rightarrow A \in \mathbb{R}^{L \times L}$$

$$Q, K, V \in \mathbb{R}^{L \times d}$$

$$A = QK^T \in \mathbb{R}^{L \times L} \text{ — скалярное произведение векторов}$$

To compute relevance scores we use two linear transformations of the input:

- Q — queries
- K — keys

And to compute final embeddings we use the third one — V — values

1.1.5 Self-attention step by step

W^Q, W^K, W^V — trainable parameters

$$q_i = x_i W^Q$$

$$k_i = x_i W^K$$

$$v_i = x_i W^V$$

- Compute queries, keys, values, for all inputs using the same linear transformations

For token i:

- Compute relevance scores using query i and all the keys
- Compute weighted combination of values, weights from relevance scores

1.1.6 Self-attention in matrix form

Q, K, V — linear

-Step 1

$$X \times W^Q = Q \quad X \times W^K = K \quad X \times W^V = V$$

-Step 2

$$\text{Attention score: } A = \text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V = Z$$

$$\text{Attention (Q, K, V)} = \text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$

1.1.7 Attention matrix

Scaled dot-product attention

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

1.1.8 Other score functions

Name	Alignment score function	Citation
Content-base attention	$\text{score}(s_t, \mathbf{h}_i) = \text{cosine}[s_t, \mathbf{h}_i]$	Graves2014
Additive(*)	$\text{score}(s_t, \mathbf{h}_i) = \mathbf{v}_a^T \tanh(\mathbf{W}_a[s_t; \mathbf{h}_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a s_t)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(s_t, \mathbf{h}_i) = s_t^T \mathbf{W}_a \mathbf{h}_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer	Luong2015
Dot-Product	$\text{score}(s_t, \mathbf{h}_i) = s_t^T \mathbf{h}_i$	Luong2015
Scaled DotProduct(^)	$\text{score}(s_t, \mathbf{h}_i) = \frac{s_t^T \mathbf{h}_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

1.1.9 Complexity

$$L \rightarrow n$$

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types, n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity Layer	per Sequential Operations	Opera- tions	Maximum Length	Path
Self attention	$O(n^2 \cdot d)$	$O(1)$		$O(1)$	
Recurrent	$O(n \cdot d^2)$	$O(n)$		$O(n)$	
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$		$O(\log_k(n))$	
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$		$O(n/r)$	

1.1.10 Multi-Head Attention — idea

Want to consider different types of relevance

- (a) Single-head attention
- (b) Multi-head attention

1.1.11 Multi-Head Attention

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^0$.

$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Use several attention heads — each will learn its own type of relevance

1.1.12 Multi-Head Attention — step by step

1) This is our input sentence*

2) We embed each word*

* In all encoders other than N0, we don't need embedding. We start directly with the output of the encoder right below this one

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using of the resulting Q/K/V matrices

5) Concatenate the resulting 2 matrices, then multiply with weight matrix W^0 to produce the output of the layer

1.1.13 Multi-Head Attention

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^0$.

$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Trainable parametrs:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

$$W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

$$d_k = d_v = d_{\text{model}}/h$$

$$d' = \frac{d}{h}$$

h — num heads

1.2 Transformer Model

1.2.1 Transformer model overview

Task — neural machine translation (Seq2seq task)

- Encoder–decoder architecture
- 3 types of attention operators:
 - Encoder–encoder (self–attention)
 - Decoder–decoder (self–attention)
 - Encoder–decoder
- Other operations (position–wise):
 - Feed forward layers
 - Residual connections
 - Layer norm
- Positional encoding

1.2.2 Encoder–encoder attention

The one we have discussed earlier:

- Multi–head
- Scaled dot–product
- Self–attention: Q, K, V are computed from the same input matrix X

Update embeddings of tokens from the input sequence

1.2.3 Decoder–decoder attention

Masked Decoder Self–Attention

We need mask attention matrix to not look at future tokens during training:

apply mask $W'_{ij} \leftarrow -\infty$ if $j > i$

$\text{softmax } W = \text{softmax}(W')$

Attend to the previous in the generated output sequence

1.2.4 Encoder–decoder attention

Encoder $\rightarrow W_K W_V$

This is not self–attention:

- Q from decoder
- K, V from encoder

$$Q \in \mathbb{R}^{L_D \times d}$$

$$K \in \mathbb{R}^{L_E \times d}$$

$$V \in \mathbb{R}^{L_E \times d}$$

Attend to tokens from the input sequence relevant for the generation of the next output token

1.2.5 Other operations

ALL other operations are position–wise:

- Feed forward layers $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \rightarrow GeLU$
- Residual connections
- Layer norm

1.2.6 Positional encoding

Attention and position–wise feed forward layers look at the input sequence as a bag of tokens $\Rightarrow$ We need to encode positional information into the input embeddings

Variant from the initial paper:

$$PE_{(\text{pos}, 2i)} = \sin(\text{pos} / 10000^{2i/d_{\text{model}}})$$

$$PE_{(\text{pos}, 2i+1)} = \cos(\text{pos} / 10000^{2i/d_{\text{model}}})$$

Other common variant–trainable vectors

1.2.7 Transformer size

- Number of layers (blocks) in encoder/decoder
- Hidden dimensionality (d_{model})
- Number of heads

- Dimensionality of inner feed-forward layers (usually $4 \times$ hidden dimensionality)

Standard model; 6+6 layers, H=512, 8 heads, 65M params

1.2.8 Training details

- Loss — standard NLL (cross-entropy) — lower is better: $NLL(y_{1:M}) = - \sum_{t=1}^M \log p(y_t | t_{t-1})$ Instead perplexity is usually reported — higher is better: $\text{Perplexity}(y_{1:M}) = 2^{\frac{1}{M} NLL(y_{1:M})}$
- Teacher forcing for decoder
- Adam optimizer
- Learning rate schedule with warm-up: $lr = d_{\text{model}}^{-0.5} \cdot \min(\text{step-num}^{-0.5}, \text{step-num} \cdot \text{warmup}^{-1.5})$

1.2.9 Training details

- Label smoothing $y_{ls} = (1 - \alpha) \cdot y_{hot} + \alpha/K$
- Residual dropout — to the output of each sub-layer(before add+norm) + to the sums of the embeddings and the positional encodings
- BPE/Word-piece (shared embeddings for input/output)
- Model averaging (average last k checkpoints — SWA)

1.2.10 Output generation: greedy search

$$p(x) = p(x_0)p(x_1|x_0) \dots p(x_t|\dots)$$

How to find the most probable output sequence $y_1, \dots, y_T$?

Greedy search: $y_t = \arg\max_{y \in \gamma} P(y|y_1, \dots, y_{t-1}, C)$

Result of greedy search $P = 0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$

Better output sequence $P = 0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$

1.2.11 Output generation: beam search

- At step choose k best candidates ($k \approx 5$)
- Stop when k candidates with $\langle \text{END} \rangle$ token have been generated
- Choose the best one: $\frac{1}{L^\alpha} \log P(y_1, \dots, y_L | C)$ length penalty, $\alpha=0.75$

1.2.12 BLEU metric for NMT

Compare NMT output and reference translation based of the n-gram overlap:

N-gram precision: $p_n = \frac{\sum_{n\text{-gram} \in \text{hyp}} \text{count}_{clip}(n\text{-gram})}{\sum_{n\text{-gram} \in \text{hyp}} \text{count}(n\text{-gram})}$ $\text{count}_{clip} = \min(\text{count}, \text{max-ref-count})$
n=1-4

Brevity penalty:

$$\text{BP} = \begin{cases} e^{(1 - |\text{ref}|/|\text{hyp}|)} & \text{if } |\text{ref}| > |\text{hyp}| \\ 1 & \text{otherwise} \end{cases}$$

Result (higher is better): $BLEU = BP \cdot \exp \left[\frac{1}{N} \sum_{n=1}^N \log p_n \right]$

1.3 Metrics and Inference for sequence generation

1.4 Transformer-based LMs and contextualized word embeddings: ELMO, BERT, GPT

1.5 Transformer for Computer Vision